Java OOP + String Practice Problem

Employee Profile & Resume Analyzer

Problem Statement

A software company wants to develop an Employee Profile and Resume Analyzer System. The system should store employee information and automatically generate a professional employee profile by performing different String operations.

1. Employee Class

Create a class named **Employee** with the following private data members:

```
employeeId
fullName
email
phoneNumber
designation
department
skills
about
```

Create a parameterized constructor and appropriate getter/setter methods.

Sample Input

```
Employee ID : EMP20261025
Full Name   : "  Rahul Kumar Sharma  "
Email       : " Rahul.Sharma@Example.COM "
Phone       : "98765-43210"
Designation : "Software Engineer"
Department  : "Java Development"
Skills      : "Java, Spring Boot, SQL, React, Docker"
About       : "Java developer with experience in building scalable web applications"
```

2. Name Processing

Clean the employee's name by removing unnecessary spaces. Then perform the following operations:

- Convert the name into uppercase.
- Convert the name into lowercase.
- Find the length of the name.
- Find the first character.
- Find the last character.
- Extract the first name.
- Extract the last name.
- Extract the middle name.
- Count the number of words.
- Generate employee initials.

```
For: Rahul Kumar Sharma
Initials: RKS
```

3. Email Processing

Clean the email and convert it to lowercase. For **Rahul.Sharma@Example.COM**, generate **rahul.sharma@example.com**.

- Find the position of @.
- Extract the username.
- Extract the domain.
- Check whether the email contains @.
- Check whether the email ends with .com.
- Check whether the email belongs to example.com.

- Compare the email domain using equalsIgnoreCase().

4. Phone Number Processing

For **98765-43210**, remove the '-' and generate **9876543210**.

- Find its length.
- Extract the first three digits.
- Extract the last four digits.
- Check whether the phone number has exactly 10 digits.

5. Designation Processing

For the designation **Software Engineer**:

- Convert it to uppercase.
- Convert it to lowercase.
- Check whether it starts with "Software".
- Check whether it ends with "Engineer".
- Check whether it contains "Engineer".
- Replace the space with "-".

```
Expected: Software-Engineer
```

6. Skills Processing

For **Java, Spring Boot, SQL, React, Docker**:

- Check whether the employee knows Java.
- Check whether the employee knows React.
- Check whether the employee knows Python.
- Find the position of the first comma.
- Extract the first skill.
- Extract the last skill.
- Count the number of skills.
- Replace commas with " | ".

```
Expected: Java | Spring Boot | SQL | React | Docker
```

7. About / Resume Processing

For the about section, perform:

- Find the length of the description.
- Count the number of words.
- Check whether it contains "Java".
- Check whether it contains "Python".
- Replace "Java" with "Full Stack Java".
- Convert the complete description to uppercase.
- Extract the first 30 characters as a short profile summary.

8. Employee ID Processing

For **EMP20261025**:

- Check whether the ID starts with "EMP".
- Extract "EMP".
- Extract the numeric portion.
- Find the length of the ID.
- Generate a new employee code using the first three characters and last four digits.

```
Expected: EMP1025
```

9. Generate Professional Profile

Create the following method:

```
public void displayProfile()
```

It should generate output similar to:

```
========== EMPLOYEE PROFILE ==========

Employee ID     : EMP20261025
Name            : Rahul Kumar Sharma
Initials        : RKS
Email           : rahul.sharma@example.com
Email Username  : rahul.sharma
Email Domain    : example.com
Phone           : 9876543210
Designation     : Software Engineer
Department      : JAVA DEVELOPMENT

Skills:
Java | Spring Boot | SQL | React | Docker

About:
Java developer with experience in building scalable web applications

=======================================
```

10. OOP Extension

Create two subclasses:

```
class Developer extends Employee
class Manager extends Employee
```

Add the following additional fields:

```
Developer → programmingLanguage
Manager → teamSize
```

Override **displayProfile()** so that developers and managers display their additional information.

Also create **getEmployeeType()** and override it in both subclasses.

String Methods Students Should Practice

```
length()
charAt()
substring()
indexOf()
lastIndexOf()
equals()
equalsIgnoreCase()
contains()
startsWith()
endsWith()
toUpperCase()
toLowerCase()
trim()
replace()
concat()
```

Important Restriction

Do not use String.split() initially. Students should practice indexOf(), substring(), charAt(), and character traversal to solve the problem.

OOP Concepts Covered

- Class & Object
- Private fields
- Encapsulation
- Constructors
- Getters/Setters
- Inheritance
- Method Overriding
- Polymorphism
- super
- Method design